CHAPTER 14

Designing APIs with Lifetimes

The previous chapters explained what lifetimes mean and how the compiler checks them. This one is about choosing them. Every reference in a public signature is a promise, and the signature is the contract: your function's body is checked against it once, but every caller is checked against it at every call, for as long as the API exists. A lifetime that is easy to write can be hard to live with. A struct with three lifetime parameters compiles today; tomorrow every type that contains it needs three lifetime parameters too, and none of them can be sent to a thread.

Good API design with lifetimes is mostly about *where* to put them. Borrowing is what makes Rust fast: a tokenizer that hands out slices of its input allocates nothing. Owning is what makes Rust values easy to move around: a report that owns its strings can be stored, returned, cached or sent anywhere. The art is to borrow where the work happens and to own where the results are kept, and to keep the remaining lifetimes few, honest and well named.

In this chapter you will learn:

- how to choose between borrowing and owning for parameters, return values and fields, and what `impl AsRef<str>` and `impl Into<String>` add;
- when `Cow<'a, str>`, `Arc<str>` or `Rc<str>` is a better answer than a lifetime parameter;
- how to build a zero-copy tokenizer with a borrowed token type, and an owned twin for storage;
- how many lifetime parameters a struct needs, and how to name them;
- how to recognize and fix the `&'a mut T<'a>` "borrowed forever" antipattern;
- how lifetimes shape callbacks, iterators that return references, errors and builders;
- how to refactor "lifetime soup" into a design with few, well-named lifetimes.

14.1 Own or borrow?

For every reference you are about to write in a signature, ask one question: does the code on the other side need the data *after the call returns*? If not, borrow. If it does, own. "Borrow for the call, own for storage" settles most cases; the rest of this section applies it to parameters, return values and fields. The program is `code/examples/ch14_parameters.rs`.

Parameters

A function that only reads its argument during the call should borrow it:

```rust
/// Reads its argument during the call only: borrow it.
fn count_words(text: &str) -> usize {
    text.split_whitespace().count()
}
```

`&str` is the most flexible string parameter there is. It accepts a literal, a `&String` (through deref coercion), and a slice of either. A parameter of type `&String` would accept only the second; the same goes for `&[T]` versus `&Vec<T>`. The lifetime is elided, and no caller ever has to think about it: the borrow lasts for the call, and every borrow a caller can produce lasts at least that long.

When callers hold many different string types, a generic parameter can accept them all, by value or by reference:

```rust
/// Also reads only, but accepts anything that can lend a `&str`, by value or by
/// reference.
fn count_lines(text: impl AsRef<str>) -> usize {
    text.as_ref().lines().count()
}
```

`impl AsRef<str>` accepts `String`, `&String`, `&str`, `Cow<str>`, `Box<str>`, `Arc<str>` and more; the standard library uses the same idea for paths (`std::fs::read_to_string` takes `P: AsRef<Path>`). The function is still a borrower: it calls `as_ref()` to get a `&str` that is valid during the call. The cost is that the function is generic, so it is compiled once per argument type; for most functions plain `&str` is the better default, and `AsRef` is a convenience for APIs that are called with owned values often.

When the function *keeps* its argument, the signature should say so by taking ownership:

```rust
/// Keeps its argument, so it takes ownership. With `impl Into<String>` the
/// caller passes a literal (one allocation here) or a `String` (moved, no copy).
struct Label {
    name: String,
}

impl Label {
    fn new(name: impl Into<String>) -> Self {
        Label { name: name.into() }
    }
}
```

With `name: &str`, `Label::new` would have to call `name.to_string()`, and a caller who already had a `String` it no longer needed would pay for a copy anyway. With `impl Into<String>`, the caller decides: a literal is converted (one allocation, which was unavoidable), and a `String` is moved in without copying. Chapter 7 met the same choice from the other side: a function that needs to keep a string and asks for `&'static str` instead forces its callers to produce literals or leaked data, while one that asks for ownership accepts anything.

Return values

A function returns a reference when its result is a *part* of an input, and an owned value when it creates something new:

```rust
/// Returns a view into its input: the result borrows from `text` (elision).
fn first_word(text: &str) -> &str {
    text.split_whitespace().next().unwrap_or("")
}
```

```
/// Returns new data: the result is owned, and the signature has no lifetime.
fn shout(text: &str) -> String {
    text.to_uppercase()
}
```

```
let owned = String::from("lifetimes are regions\nnot durations");
println!("words: {}", count_words("a literal")); // `&'static str`
println!("words: {}", count_words(&owned)); // `&String` coerces to `&str`
println!("words: {}", count_words(&owned[..9])); // a slice of a `String`
println!("lines: {}", count_lines(&owned)); // a reference...
println!("lines: {}", count_lines(String::from("one line"))); // ...or a value

let fixed = Label::new("fixed"); // allocates a `String`
let built = Label::new(format!("label-{}", 2)); // moves the `String` in
println!("labels: {}, {}", fixed.name, built.name);

let first = first_word(&owned); // borrows from `owned`
println!("first: {first}, shouted: {}", shout(first));
```

```
words: 2
words: 5
words: 1
lines: 2
lines: 1
labels: fixed, label-2
first: lifetimes, shouted: LIFETIMES
```

`first_word` returns a view: by elision (Chapter 3) the result borrows from `text`, so `first` keeps `owned` borrowed for as long as `first` is used. That is the right contract for a view, and it costs nothing at run time. `shout` creates a new string, so it returns a `String` and its signature has no lifetime at all. A function cannot return a reference to something it created (E0515, Chapter 5), so the choice is usually made for you; the interesting case is a function whose result is *sometimes* a part of the input and sometimes new, which is what `Cow` is for (next section).

Fields

A struct that is kept, such as a configuration, a record or a result, should own its fields. It then has no lifetime parameter, and it can be stored in a collection, returned from any function, cached, or sent to another thread.

A struct with a reference field is a *view*: it has a lifetime parameter (Chapter 2), it is cheap to create, and it can never outlive the data it borrows. The lifetime also spreads. Every struct that contains a `View<'a>` needs a lifetime parameter of its own, every function that returns one ties its result to an input, and none of them satisfies `T: 'static` unless it borrows `'static` data (Chapter 7). Views are the right tool for short-lived things: tokenizers, iterators, parsed tokens, the result of a lookup. They are the wrong tool for anything that is meant to be kept.

Situation	Choose
read during the call only	`&str`, `&[T]`, `&T`
read during the call, many argument types	`impl AsRef<str>`
kept after the call	`String`, `Vec<T>`, `impl Into<String>`
return a part of an input	`&str`, tied to that input
return new data	`String`
return a part or new data	`Cow<'a, str>`
a short-lived view	a struct with a lifetime parameter
a value that is stored, shared or sent	owned fields, `Arc<str>`

14.2 Alternatives to a lifetime parameter

Between "borrow" and "own" there are types that do a bit of both. They let you avoid a lifetime parameter, or avoid a copy, in places where neither plain choice fits. The programs are `code/examples/ch14_cow.rs` and `code/examples/ch14_shared_strings.rs`.

`Cow<'a, str>`: borrow when you can, own when you must

`std::borrow::Cow` ("clone on write") is an enum with two variants. The real definition is generic over any type that can be borrowed and owned; for strings, `Cow<'a, str>` amounts to this (not compiled, and not the real definition):

```
enum CowStr<'a> {
    Borrowed(&'a str), // `Cow::Borrowed`
    Owned(String),     // `Cow::Owned`
}
```

It dereferences to `&str` in both cases, so the caller can use it like a string. It shines as the return type of a function that usually returns its input unchanged but sometimes has to build a new string:

```
/// Escapes the three characters that are special in markup. Most text contains
/// none of them, and then the result borrows the input.
fn escape(input: &str) -> Cow<'_, str> {
    if !input.contains(['<', '>', '&']) {
        return Cow::Borrowed(input);
    }
    let mut escaped = String::with_capacity(input.len() + 8);
    for c in input.chars() {
        match c {
            '<' => escaped.push_str("&lt;"),
            '>' => escaped.push_str("&gt;"),
            '&' => escaped.push_str("&amp;"),
            _ => escaped.push(c),
        }
    }
    Cow::Owned(escaped)
}
```

```
for input in ["plain text", "a < b && c"] {
    let escaped = escape(input);
    println!("{:<9} {escaped}", describe(&escaped));
}
```

```
let kept: String = escape("x > 0").into_owned(); // detach from the input
println!("kept      {kept}");
```

```
borrowed  plain text
owned     a &lt; b &amp;amp;&amp;amp; c
kept      x &gt; 0
```

Most text contains no `<`, `>` or `&`, and for that text `escape` returns the input itself, without allocating. The signature, `fn escape(input: &str) -> Cow<'_, str>`, tells the caller the truth: the result may borrow from `input`. A caller who wants to keep the result after `input` is gone calls `into_owned()`, which returns the `String`, either the one already inside or a fresh copy of the borrowed text.

`Cow<'static, str>` in a field

With `'static` as the lifetime, `Cow` becomes a field type that holds either a literal or a run-time string, and has no lifetime parameter to spread:

```
/// A column name is usually a literal, sometimes built at run time.
#[derive(Debug)]
struct Column {
    name: Cow<'static, str>,
}

impl Column {
    fn new(name: impl Into<Cow<'static, str>>) -> Self {
        Column { name: name.into() }
    }
}
```

```
let columns = vec![Column::new("id"), Column::new(format!("score_{}", 2026))];
for column in &columns {
    println!("{:<9} {}", describe(&column.name), column.name);
}
// `Column: 'static`, so the columns can move to another thread.
let handle = thread::spawn(move || columns.len());
println!("{} columns sent to a thread", handle.join().unwrap());
```

```
borrowed  id
owned     score_2026
2 columns sent to a thread
```

`Column::new("id")` stores the literal (no allocation); the formatted name is stored as an owned `String`. Both kinds of `Column` are the same type, and since `Cow<'static, str>` holds no borrow that can expire, `Column: 'static`: the vector moves to another thread without complaint. This is rung 3 of the decision ladder in Chapter 7, "mix". It fits well for names, labels and messages that are usually literals.

`Arc<str>` and `Rc<str>`: shared, immutable text

When the same immutable string is needed in many places, such as a name that is both a field of a record and a key of an index, copying it into each place wastes memory, and borrowing it from one place ties every other place to it with a lifetime. `Arc<str>` is the third option: a reference-counted string that several owners share. Cloning it copies a pointer and increments a counter.

```
/// A file entry. Its name is also a key of the index below.
struct Entry {
```

```
    name: Arc<str>,
    size: u64,
}

/// Entries plus an index by name, sharing one allocation per name.
struct Catalog {
    entries: Vec<Entry>,
    by_name: BTreeMap<Arc<str>, usize>,
}

impl Catalog {
    fn new(files: &[(&str, u64)]) -> Self {
        let mut catalog = Catalog {
            entries: Vec::new(),
            by_name: BTreeMap::new(),
        };
        for &(name, size) in files {
            let name: Arc<str> = Arc::from(name); // one allocation
            let index = catalog.entries.len();
            catalog.by_name.insert(Arc::clone(&name), index); // a second owner
            catalog.entries.push(Entry { name, size });
        }
        catalog
    }

    fn size_of(&self, name: &str) -> Option<u64> {
        // `Arc<str>: Borrow<str>`, so a `&str` is enough to look up a key.
        self.by_name.get(name).map(|&i| self.entries[i].size)
    }
}
```

```
let catalog = Catalog::new(&[("notes.txt", 120), ("todo.md", 30)]);
println!("todo.md: {:?} bytes", catalog.size_of("todo.md"));
let first = &catalog.entries[0].name;
println!("{first}: {} owners", Arc::strong_count(first)); // entry + index

// No lifetime parameter, and `'static`: the names can go to a thread.
let names: Vec<Arc<str>> = catalog.entries.iter().map(|e| e.name.clone()).collect();
let handle = thread::spawn(move || names.join(", "));
println!("from a thread: {}", handle.join().unwrap());

// `Rc<str>` is the same idea for data that stays on one thread.
let tag: Rc<str> = Rc::from("draft");
let tags = [Rc::clone(&tag), Rc::clone(&tag)];
println!("{}: {} owners", tags[1], Rc::strong_count(&tag));
```

```
todo.md: Some(30) bytes
notes.txt: 2 owners
from a thread: notes.txt, todo.md
draft: 3 owners
```

Each name is allocated once, by `Arc::from(name)`, and shared by the entry and the index (the strong count is 2). Lookups take a plain `&str`, because `Arc<str>` implements `Borrow<str>`. Neither `Entry` nor `Catalog` has a lifetime parameter, and `Arc<str>` is `Send` and `'static`, so names can move to another thread. `Rc<str>` is the same idea without the atomic counter, for data that stays on one thread (it is not `Send`). Prefer `Arc<str>` to `Arc<String>`: it is one allocation instead of two, and the text cannot be changed anyway.

Choosing among them

Type	Lifetime parameter	Cost of a clone	`'static`
`&'a str`	yes	copies a pointer	only if `'a` is
`String`	no	copies the text	yes
`Cow<'a, str>`	yes (none with `'static`)	copies if owned	if `'a` is `'static`
`Arc<str>`, `Rc<str>`	no	increments a counter	yes

14.3 Zero-copy parsing: borrowed tokens, owned tokens

A tokenizer produces many small strings. If each token owned its text, every identifier and number would cost an allocation and a copy. If each token borrows its text from the input, producing a token costs nothing. The price is the usual one: a borrowed token cannot outlive the input. The standard design keeps both options open: a borrowed token type for the work, and an owned twin for the tokens you keep. The program is `code/examples/ch14_tokenizer.rs`.

The borrowed token

```
/// A token whose text is a slice of the source it was read from.
#[derive(Debug, Clone, Copy, PartialEq)]
enum Token<'src> {
    Ident(&'src str),
    Number(&'src str),
    Str(&'src str), // the contents, without the quotes
    Punct(char),
}
```

```
struct Tokenizer<'src> {
    rest: &'src str,
}

impl<'src> Tokenizer<'src> {
    fn new(src: &'src str) -> Self {
        Tokenizer { rest: src }
    }
}

impl<'src> Iterator for Tokenizer<'src> {
    type Item = Token<'src>; // items borrow from the source, not from the tokenizer

    fn next(&mut self) -> Option<Token<'src>> {
        let rest: &'src str = self.rest.trim_start(); // copy the `&'src str` out
        let first = rest.chars().next()?;
        let word_end =
            |pred: fn(char) -> bool| rest.find(|c| !pred(c)).unwrap_or(rest.len());
        let (token, len) = if first.is_alphabetic() || first == '_' {
            let len = word_end(|c| c.is_alphanumeric() || c == '_');
            (Token::Ident(&rest[..len]), len)
        } else if first.is_ascii_digit() {
            let len = word_end(|c| c.is_ascii_digit());
            (Token::Number(&rest[..len]), len)
        } else if first == '"' {
            let close = 1 + rest[1..].find('"')?; // an unterminated string ends the input
            (Token::Str(&rest[1..close]), close + 1)
        } else {
            (Token::Punct(first), first.len_utf8())
        };
        self.rest = &rest[len..];
```

```
        Some(token)
    }
}
```

Two details matter. First, `type Item = Token<'src>`: the items borrow from the *source*, not from the tokenizer, so a caller can collect them, keep them after the tokenizer is gone, and hold several at once (Chapter 2 explains why `next(&mut self)` may return a `Token<'src>`). Second, the first line of `next` copies the `&'src str` out of `self` into a local `rest`. All the slices are taken from that local, so they are `&'src str` and do not borrow `self`.

```
let source = String::from(r#"let total = price * 3 + "tax";"#);
let tokens: Vec<Token<'_>> = Tokenizer::new(&source).collect();
println!("{tokens:?}");

// Zero-copy: the text of every token points into `source`.
let range = source.as_bytes().as_ptr_range();
for token in &tokens {
    if let Token::Ident(text) | Token::Number(text) | Token::Str(text) = token {
        assert!(range.contains(&text.as_ptr()));
    }
}
```

```
[Ident("let"), Ident("total"), Punct('='), Ident("price"), Punct('*'), Number("3"),
Punct('+'), Str("tax"), Punct(';')]
```

The `assert!` checks the "zero-copy" claim: the text of every token points into the bytes of `source`.

When the input goes away

Borrowed tokens are a problem as soon as the input is temporary. A typical case is reading a file line by line into one reused buffer, as `BufRead::read_line` does, and keeping the tokens of every line. The program `code/compile_fail/ch14_tokens_outlive_buffer.rs` tries it with a smaller tokenizer:

```
pub fn caller(input: &[&str]) -> usize {
    let mut line = String::new();
    let mut history: Vec<Token<'_>> = Vec::new();
    for text in input {
        line.clear(); // needs `&mut line`...
        line.push_str(text);
        history.extend(tokenize(&line)); // ...but `history` borrows `line`
    }
    history.len()
}
```

The compiler rejects it with two errors, one for `line.clear()` and one for `line.push_str(text)`; here is the first (the second is identical in shape and omitted):

```
error[E0502]: cannot borrow `line` as mutable because it is also borrowed as immutable
  --> compile_fail/ch14_tokens_outlive_buffer.rs:19:9
   |
19 |         line.clear(); // needs `&mut line`...
   |         ^^^^^^^^^^^^ mutable borrow occurs here
20 |         line.push_str(text);
21 |         history.extend(tokenize(&line)); // ...but `history` borrows `line`
   |         -------                ----- immutable borrow occurs here
   |         |
   |         immutable borrow later used here
```

Read the labels in the order of execution, across two iterations of the loop. In the first iteration, `&line` is borrowed ("immutable borrow occurs here") and the tokens holding that borrow are stored in `history`. In the second iteration, `line.clear()` needs a mutable borrow ("mutable borrow occurs here"), while the first iteration's borrow is still alive, because `history` is used again on the same line ("immutable borrow later used here"). Clearing the buffer would destroy the text that the stored tokens point to.

The owned twin

The fix is to convert the tokens you keep into a type that owns its text:

```
/// The same token, owning its text: it can outlive the source.
#[derive(Debug, Clone, PartialEq)]
enum OwnedToken {
    Ident(String),
    Number(String),
    Str(String),
    Punct(char),
}

impl Token<'_> {
    /// Copies the borrowed text, so the result no longer depends on the source.
    fn into_owned(self) -> OwnedToken {
        match self {
            Token::Ident(s) => OwnedToken::Ident(s.to_owned()),
            Token::Number(s) => OwnedToken::Number(s.to_owned()),
            Token::Str(s) => OwnedToken::Str(s.to_owned()),
            Token::Punct(c) => OwnedToken::Punct(c),
        }
    }
}
```

```
// Reading line by line into one reused buffer, as `BufRead::read_line` does.
let input = ["x = 1", "y = x + 2"];
let mut line = String::new();
let mut history: Vec<OwnedToken> = Vec::new();
for text in input {
    line.clear();
    line.push_str(text);
    history.extend(Tokenizer::new(&line).map(Token::into_owned));
}
let last = history.last();
println!("kept {} tokens, the last is {last:?}", history.len());
```

```
kept 8 tokens, the last is Some(Number("2"))
```

The tokenizer is unchanged and still allocates nothing; the copies happen only for the tokens that are kept, at the point where they are kept. The method is called `into_owned`, after `Cow::into_owned`. It cannot be called `to_owned`: `Token` derives `Clone`, so it already has a `to_owned` method (from the blanket `ToOwned` implementation for `Clone` types), and that one returns another `Token<'src>`.

Two other designs are worth knowing. A token can hold `Cow<'src, str>` fields, so that one type covers both cases and the owned version is `Token<'static>` (exercise 1). Or a token can store a byte range (`Range<usize>`) into the input instead of a slice: it then has no lifetime at all, and whoever still has the input can slice it again. The error type later in this chapter uses the second idea.

14.4 How many lifetime parameters does a struct need?

Most borrowing structs need exactly one lifetime parameter, even when they hold several references. One parameter can serve several fields because shared references are covariant (Chapter 4): when you pass two borrows with different lifetimes, the compiler picks the shorter one for the parameter, and the longer borrow is shortened to fit. The struct is then usable while *both* borrows are valid, which is usually all you need.

"Usually" is the important word. The program `code/compile_fail/ch14_one_lifetime.rs` defines a field splitter with one lifetime parameter for the line and the separator:

```
pub struct Fields<'a> {
    rest: Option<&'a str>,
    sep: &'a str, // the same lifetime as the line
}

impl<'a> Fields<'a> {
    pub fn new(line: &'a str, sep: &'a str) -> Self {
        Fields {
            rest: Some(line),
            sep,
        }
    }
}
```

Its `Iterator` implementation returns `&'a str` fields, slices of the line. Now a convenience function builds the separator as a local `String` and returns the first field:

```
pub fn first_field(line: &str, sep: char) -> &str {
    let sep = sep.to_string();
    Fields::new(line, &sep).next().unwrap_or("")
}
```

```
error[E0515]: cannot return value referencing local variable `sep`
  --> compile_fail/ch14_one_lifetime.rs:42:5
   |
42 |     Fields::new(line, &sep).next().unwrap_or("")
   |     ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
   |     |                  |
   |     |                  `sep` is borrowed here
   |     returns a value referencing data owned by the current function
```

"`sep` is borrowed here", and the value returned is "referencing data owned by the current function". The field returned is a slice of `line`, which outlives the call, but the type does not say so. With a single parameter, `'a` had to be short enough for *both* arguments, so it ended when the local `sep` is dropped, and the item type `&'a str` inherited that limit. The struct forgot which of its two borrows the results come from.

Giving each borrow its own parameter keeps that information (`code/examples/ch14_two_lifetimes.rs`):

```
/// Splits a line into fields at a separator. The fields are slices of the
/// line; the separator is only consulted.
struct Fields<'line, 'sep> {
    rest: Option<&'line str>,
    sep: &'sep str,
}

impl<'line, 'sep> Fields<'line, 'sep> {
```

```
    fn new(line: &'line str, sep: &'sep str) -> Self {
        assert!(!sep.is_empty(), "the separator must not be empty");
        Fields {
            rest: Some(line),
            sep,
        }
    }
}

impl<'line> Iterator for Fields<'line, '_> {
    type Item = &'line str; // fields borrow from the line only

    fn next(&mut self) -> Option<&'line str> {
        let rest = self.rest?;
        match rest.find(self.sep) {
            Some(i) => {
                self.rest = Some(&rest[i + self.sep.len()..]);
                Some(&rest[..i])
            }
            None => {
                self.rest = None;
                Some(rest)
            }
        }
    }
}
```

```
/// The separator is built inside the function and dropped at its end; the
/// result may borrow only from `line`.
fn first_field(line: &str, sep: char) -> &str {
    let sep = sep.to_string();
    Fields::new(line, &sep).next().unwrap_or("")
}
```

```
let record = "ada;lovelace;1815";
let fields: Vec<&str> = Fields::new(record, ";").collect();
println!("{fields:?}");
println!("{:?}", first_field(record, ';'));
```

```
["ada", "lovelace", "1815"]
"ada"
```

The item type mentions only `'line`, and the `Iterator` implementation writes the separator's lifetime as `'_` (Chapter 6), because nothing in it depends on that lifetime. So `first_field` may return a field that outlives its local separator.

The rule of thumb: start with one lifetime parameter, and add a second one when

- the struct borrows from two sources and its results borrow from only one of them, which may outlive the other (as here); or
- the struct holds a `&mut` to something that itself holds borrows (the next section shows why).

Otherwise, one is enough, and every extra parameter is something every user of the struct has to carry.

Name lifetimes after what they borrow

`'a` and `'b` say nothing. `'line` and `'sep` say which borrow is which, and so do `'src`, `'text`, `'doc` and `'out` in this chapter. The standard library does the same in `thread::Scope<'scope, 'env>`, and serde's `Deserialize<'de>` names the lifetime of the data being deserialized (Chapter 9). Diagnostics benefit too: a message such as "lifetime `'src` defined here" points you at the right borrow immediately.

A single `'a` is fine in a short generic function with one borrow; structs, traits and anything public deserve names.

14.5 The `&'a mut T<'a>` antipattern: borrowed forever

Chapter 2 introduced this trap, Chapter 3 met it as the consequence of a tempting elision rule ("make every lifetime the same"), and Chapter 4 explained the invariance behind it. Here it is as an API design mistake, because that is where it usually appears: in a method that seemed to need it. The program `code/compile_fail/ch14_borrowed_forever.rs` collects the distinct words of a text:

```
pub struct WordSet<'text> {
    words: Vec<&'text str>,
}

impl<'text> WordSet<'text> {
    pub fn new() -> Self {
        WordSet { words: Vec::new() }
    }

    pub fn add_all(&'text mut self, text: &'text str) {
        for word in text.split_whitespace() {
            if !self.words.contains(&word) {
                self.words.push(word);
            }
        }
    }

    pub fn len(&self) -> usize {
        self.words.len()
    }
}

pub fn caller() -> usize {
    let text = String::from("the cat saw the dog");
    let mut set = WordSet::new();
    set.add_all(&text);
    set.len() // the set is still borrowed by `add_all`
}
```

The type compiles, and so does `add_all`. The caller does not:

```
error[E0502]: cannot borrow `set` as immutable because it is also borrowed as mutable
  --> compile_fail/ch14_borrowed_forever.rs:33:5
   |
32 |     set.add_all(&text);
   |     --- mutable borrow occurs here
33 |     set.len() // the set is still borrowed by `add_all`
   |     ^^^
   |     |
   |     immutable borrow occurs here
   |     mutable borrow later used here
```

The labels contain the hallmark of this antipattern. "Mutable borrow occurs here" points at `add_all`, and at the very next use of `set`, the compiler reports both "immutable borrow occurs here" and "mutable borrow later used here", on the same expression. The mutable borrow taken by `add_all` is still alive at every later use of `set`, so *any* later use conflicts with it.

Why? `add_all` takes `&'text mut WordSet<'text>`: the borrow of the set and the words inside it have the same lifetime. `WordSet<'text>` sits behind `&mut`, where it is invariant (Chapter 4), so `'text` cannot

be shortened to make the borrow short. The borrow of `set` therefore lasts for all of `'text`, and `'text` must cover every point where `set` is used, since `set`'s type mentions it. After the first call, the set is mutably borrowed for the rest of its life.

The author probably wrote `&'text mut self` because `add_all` stores `&'text str` values in the set. But storing them requires only that the *argument* lives for `'text`, not the borrow of the set. The fix, in `code/examples/ch14_borrowed_forever_fixed.rs`, lets the receiver's borrow have its own, anonymous lifetime:

```
struct WordSet<'text> {
    words: Vec<&'text str>,
}

impl<'text> WordSet<'text> {
    fn new() -> Self {
        WordSet { words: Vec::new() }
    }

    fn add_all(&mut self, text: &'text str) {
        for word in text.split_whitespace() {
            if !self.words.contains(&word) {
                self.words.push(word);
            }
        }
    }

    fn len(&self) -> usize {
        self.words.len()
    }
}
```

```
let text = String::from("the cat saw the dog");
let mut set = WordSet::new();
set.add_all(&text);
println!("{} distinct words", set.len());
set.add_all("the end"); // a second mutable borrow: fine now
println!("{} distinct words: {:?}", set.len(), set.words);
```

```
4 distinct words
5 distinct words: ["the", "cat", "saw", "dog", "end"]
```

Now each call borrows the set only for the call, and the set can be used and mutated again afterward.

The same shape in a field

The antipattern also hides in struct fields. A `Collector` that pushes words into a vector owned by someone else is natural to write with one lifetime (`code/compile_fail/ch14_collector_forever.rs`):

```
/// Pushes words into a vector owned by someone else.
pub struct Collector<'a> {
    out: &'a mut Vec<&'a str>,
}

impl<'a> Collector<'a> {
    pub fn push(&mut self, word: &'a str) {
        self.out.push(word);
    }
}

pub fn caller() -> usize {
    let text = String::from("to be or not to be");
```

```
    let mut words: Vec<&str> = Vec::new();
    {
        let mut collector = Collector { out: &mut words };
        for word in text.split_whitespace() {
            collector.push(word);
        }
    } // the collector is gone, but `words` is still mutably borrowed
    words.len()
}
```

```
error[E0502]: cannot borrow `words` as immutable because it is also borrowed as mutable
  --> compile_fail/ch14_collector_forever.rs:26:5
   |
21 |         let mut collector = Collector { out: &mut words };
   |                                              ---------- mutable borrow occurs here
...
26 |     words.len()
   |     ^^^^^
   |     |
   |     immutable borrow occurs here
   |     mutable borrow later used here
```

The field type `&'a mut Vec<&'a str>` is `&'a mut T<'a>` again: the borrow of `words` and the strings inside it share `'a`, and `Vec<&'a str>` is invariant behind `&mut`. The strings must stay valid as long as `words` is used, so the mutable borrow of `words` must too, even after the collector is gone. Here a struct needs two lifetime parameters, one for each borrow (`code/examples/ch14_collector.rs`):

```
/// `'out`: how long we may write into the vector.
/// `'text`: how long the words we push stay valid.
struct Collector<'out, 'text> {
    out: &'out mut Vec<&'text str>,
}

impl<'text> Collector<'_, 'text> {
    fn push(&mut self, word: &'text str) {
        self.out.push(word);
    }
}
```

```
let text = String::from("to be or not to be");
let mut words: Vec<&str> = Vec::new();
{
    let mut collector = Collector { out: &mut words };
    for word in text.split_whitespace() {
        collector.push(word);
    }
} // the borrow `'out` ends here; `'text` goes on
words.sort(); // `words` is ours again: we can mutate it
words.dedup();
println!("{} distinct words: {words:?}", words.len());
```

```
4 distinct words: ["be", "not", "or", "to"]
```

To spot the antipattern, look for a lifetime that appears both on a `&mut` and inside the type it points to: `&'a mut self` in `impl<'a> T<'a>`, `&'a mut T<'a>`, `&'a mut Vec<&'a str>`. The fix is always the same: give the outer borrow its own lifetime, which in a method usually means eliding it.

14.6 Callbacks

A callback parameter is a small signature inside a signature, and its lifetimes decide what the callback may do with its arguments. The program is `code/examples/ch14_callbacks.rs`.

Callbacks that are called, not stored

When a function only calls its callback before returning, take it as a generic `impl Fn...` parameter:

```
/// Calls `f` on every non-blank line. The callback must accept a borrow of any
/// lifetime (the bound is higher-ranked, Chapter 9), so it cannot keep it.
fn for_each_line(text: &str, mut f: impl FnMut(&str)) {
    for line in text.lines().filter(|line| !line.trim().is_empty()) {
        f(line);
    }
}
```

The bound `impl FnMut(&str)` has an elided lifetime in the callback's argument, which makes it higher-ranked: it means `impl for<'l> FnMut(&'l str)`, "a callback that accepts a `&str` of *any* lifetime" (Chapter 9 explains `for<'l>` in detail). That gives the function freedom. It may pass the callback slices of `text`, or borrows of a buffer it creates and drops itself, and no caller can tell the difference.

The other side of that freedom is that the callback cannot keep what it receives. The program `code/compile_fail/ch14_callback_keeps_borrow.rs` tries to collect the lines:

```
pub fn caller(text: &str) -> usize {
    let mut kept: Vec<&str> = Vec::new();
    for_each_line(text, |line| kept.push(line));
    kept.len()
}
```

```
error[E0521]: borrowed data escapes outside of closure
  --> compile_fail/ch14_callback_keeps_borrow.rs:14:32
   |
13 |     let mut kept: Vec<&str> = Vec::new();
   |         -------- `kept` declared here, outside of the closure body
14 |     for_each_line(text, |line| kept.push(line));
   |                          ----  ^^^^^^^^^^^^^^^ `line` escapes the closure body here
   |                          |
   |                          `line` is a reference that is only valid in the closure body
   |
   = note: requirement occurs because of a mutable reference to `Vec<&str>`
   = note: mutable references are invariant over their type parameter
   = help: see <https://doc.rust-lang.org/nomicon/subtyping.html> for more information about
variance
```

E0521 again, this time "borrowed data escapes outside of closure". "`line` is a reference that is only valid in the closure body": as far as the closure knows, the borrow could end as soon as the call returns. "`line` escapes the closure body here": pushing it into `kept`, which lives outside the closure. The notes explain why the compiler cannot simply pick a longer lifetime: `kept` is reached through a mutable reference, and `Vec<&str>` is invariant behind it (Chapter 4).

If the API wants callers to be able to keep the lines, it must promise that the lines come from `text`, and say so in the signature:

```
/// The same, but the signature promises that every line borrows from `text`,
/// so a callback may keep the lines for as long as `text` lives.
```

```
fn for_each_line_kept<'text>(text: &'text str, mut f: impl FnMut(&'text str)) {
    for line in text.lines().filter(|line| !line.trim().is_empty()) {
        f(line);
    }
}
```

```
let mut total = 0;
for_each_line(&text, |line| total += line.len()); // borrows `total` mutably
println!("total length: {total}");

let mut kept: Vec<&str> = Vec::new();
for_each_line_kept(&text, |line| kept.push(line)); // keeps the lines
println!("kept: {kept:?}");
```

```
total length: 40
kept: ["INFO start", "WARN disk almost full", "INFO stop"]
```

The two signatures are two different contracts. `for_each_line` promises less, so its implementation may change (to trim lines into a reused buffer, say) without breaking any caller. `for_each_line_kept` promises more, which lets callers keep the lines, but it can never be changed to pass anything that does not borrow from `text`. Choose the second only when keeping the arguments is a real use case.

Callbacks that are stored

A callback that is stored lives as long as the struct that stores it. Chapter 7 showed the default: `Box<dyn Fn(&str)>` means `Box<dyn Fn(&str) + 'static>`, so stored callbacks must own what they capture. That is the right choice for a long-lived registry. For a dispatcher that is used within one function, a lifetime parameter lets the callbacks borrow local variables:

```
/// Stores callbacks that may borrow anything that outlives `'a`.
struct Dispatcher<'a> {
    handlers: Vec<Box<dyn FnMut(&str) + 'a>>,
}

impl<'a> Dispatcher<'a> {
    fn new() -> Self {
        Dispatcher {
            handlers: Vec::new(),
        }
    }

    fn on_line(&mut self, handler: impl FnMut(&str) + 'a) {
        self.handlers.push(Box::new(handler));
    }

    fn dispatch(&mut self, line: &str) {
        for handler in &mut self.handlers {
            handler(line);
        }
    }
}
```

```
let mut count = 0;
let mut warnings = Vec::new();
{
    let mut dispatcher = Dispatcher::new();
    dispatcher.on_line(|_| count += 1);
    dispatcher.on_line(|line| {
        if line.starts_with("WARN") {
            warnings.push(line.to_owned()); // the line is borrowed: copy it
```

```
        }
    });
    for_each_line(&text, |line| dispatcher.dispatch(line));
} // the dispatcher is dropped: `count` and `warnings` are free again
println!("{count} lines, warnings: {warnings:?}");
```

```
3 lines, warnings: ["WARN disk almost full"]
```

The two closures borrow `count` and `warnings` mutably, and the dispatcher holds those borrows (`+ 'a`) until it is dropped at the end of the block. After that, both variables can be read. A `Dispatcher<'a>` cannot be stored in a global or sent to `thread::spawn`, but within its scope it accepts any closure. Note also that the second closure copies the line (`to_owned`): the dispatcher passes a `&str` that is valid only during the call, so keeping it would be the E0521 above.

14.7 Iterators that return references

A method that returns an iterator over borrowed data is easy to write with `impl Iterator`. The program is `code/examples/ch14_iterators.rs`:

```
/// The items borrow from `self`; in edition 2024 the opaque type captures
/// the elided lifetime of `&self`, so no `+ '_` is needed.
fn words(&self) -> impl Iterator<Item = &str> {
    self.body.split_whitespace()
}

fn long_words(&self, min_len: usize) -> impl Iterator<Item = &str> {
    self.words().filter(move |word| word.len() >= min_len)
}
```

The item type `&str` borrows from `&self` (elision rule 3), and the iterator itself borrows `self` as well. In edition 2024, a return-position `impl Trait` captures every lifetime in scope, including the elided lifetime of `&self`, so nothing more is needed. Edition 2021 captured only the lifetimes named in the bounds: this signature worked there too, because `&str` names the lifetime, but an iterator over the `char`s of `self.body` needed `+ '_` (error E0700 without it), which you will still see in older code. Chapter 8 has the complete rules.

An `impl Trait` type has no name, so callers cannot write it in a struct field. When callers may want to store the iterator, return a named type instead:

```
/// Returns a named type, which callers can store in a field.
fn lines(&self) -> BodyLines<'_> {
    BodyLines {
        inner: self.body.lines(),
    }
}
```

```
/// The non-blank lines of a document, trimmed.
struct BodyLines<'doc> {
    inner: std::str::Lines<'doc>,
}

impl<'doc> Iterator for BodyLines<'doc> {
    type Item = &'doc str;
```

```
    fn next(&mut self) -> Option<&'doc str> {
        self.inner
            .by_ref()
            .map(str::trim)
            .find(|line| !line.is_empty())
    }
}

/// A reader that keeps its position in a document: possible because the
/// iterator type has a name.
struct Reader<'doc> {
    lines: BodyLines<'doc>,
    line_no: usize,
}
```

`Reader<'doc>` keeps an iterator and a line counter together, which is possible only because `BodyLines<'doc>` can be named. This is why the standard library returns named types such as `Lines<'a>` and `SplitWhitespace<'a>` from `str` methods.

The capture rule of edition 2024 has one consequence for API design. When the items borrow from a *source* held by `self` rather than from `self`, capturing the lifetime of `&self` ties the iterator to `self` for no reason. Here is the problem with a temporary view, in `code/compile_fail/ch14_iterator_overcaptures.rs` (in edition 2021 this program compiles, because there the bound `Item = &'src str` decides what is captured):

```
pub struct Index<'src> {
    src: &'src str,
}

impl<'src> Index<'src> {
    pub fn words(&self) -> impl Iterator<Item = &'src str> {
        self.src.split_whitespace()
    }
}

pub fn words_of(src: &str) -> impl Iterator<Item = &str> {
    Index { src }.words()
}
```

```
error[E0716]: temporary value dropped while borrowed
  --> compile_fail/ch14_iterator_overcaptures.rs:17:5
   |
16 | pub fn words_of(src: &str) -> impl Iterator<Item = &str> {
   |                       - let's call the lifetime of this reference `'1`
17 |     Index { src }.words()
   |     ^^^^^^^^^^^^^^^^^^^^^
   |     |                   |
   |     |                   temporary value is freed at the end of this statement
   |     creates a temporary value which is freed while still in use
   |     argument requires that borrow lasts for `'1`
```

The items are `&'src str`, slices of the source, which outlives the call. But the opaque iterator type also captured the anonymous lifetime of `&self`, so it borrows the temporary `Index { src }` ("creates a temporary value which is freed while still in use"), which is dropped at the end of the statement. The fix is a *precise capture* list, `use<'src>`, which says that the iterator captures only `'src`:

```
/// A short-lived view of some source text.
struct Index<'src> {
    src: &'src str,
}
```

```
impl<'src> Index<'src> {
    /// The items borrow from the source, not from the index: `use<'src>` says
    /// that the iterator captures only `'src` (Chapter 8).
    fn words(&self) -> impl Iterator<Item = &'src str> + use<'src> {
        self.src.split_whitespace()
    }
}

/// The `Index` is a temporary, dropped at the end of the function; the
/// iterator survives it because it does not borrow it.
fn words_of(src: &str) -> impl Iterator<Item = &str> {
    Index { src }.words()
}
```

```
let doc = Document {
    body: String::from("  borrow checkers\n\n  check borrows \n"),
};
println!("{:?}", doc.words().collect::<Vec<_>>());
println!("{:?}", doc.long_words(7).collect::<Vec<_>>());

let mut reader = Reader {
    lines: doc.lines(),
    line_no: 0,
};
while let Some(line) = reader.lines.next() {
    reader.line_no += 1;
    println!("{}: {line}", reader.line_no);
}

println!("{:?}", words_of("one two three").last());
```

```
["borrow", "checkers", "check", "borrows"]
["checkers", "borrows"]
1: borrow checkers
2: check borrows
Some("three")
```

Chapter 8 explains `use<..>` in full. For API design, the point is to decide what the returned iterator borrows (`self`, or the data behind it) and to make the signature say exactly that.

14.8 Errors that borrow and errors that own

An error value describes what went wrong, and it is tempting to make it point at the offending input. The program `code/examples/ch14_errors.rs` starts with such an error:

```
/// Cheap to create, but tied to the input it points into.
#[derive(Debug)]
struct ParseError<'src> {
    line_no: usize,
    line: &'src str,
}

/// Parses `key = value` lines; the pairs and the error borrow from `src`.
fn parse(src: &str) -> Result<Vec<(&str, &str)>, ParseError<'_>> {
    let mut pairs = Vec::new();
    for (i, line) in src.lines().enumerate() {
        match line.split_once('=') {
            Some((key, value)) => pairs.push((key.trim(), value.trim())),
            None => {
                return Err(ParseError {
                    line_no: i + 1,
                    line,
```

```
                });
            }
        }
    }
    Ok(pairs)
}
```

For a caller that owns the input, `ParseError<'src>` is fine: it is cheap, and the caller can print the line. But errors travel. They are returned up the stack, converted by `?`, boxed into `Box<dyn Error>`, logged later, sent to other threads. Each of those trips makes a borrowed error harder to use. The first problem appears as soon as a function creates its own input. In `code/compile_fail/ch14_error_borrows_local.rs`, `load` reads the text and then parses it:

```
pub fn load(name: &str) -> Result<usize, ParseError<'_>> {
    let text = read_source(name);
    parse(&text) // the error would borrow `text`, which dies here
}
```

```
error[E0515]: cannot return value referencing local variable `text`
  --> compile_fail/ch14_error_borrows_local.rs:28:5
   |
28 |     parse(&text) // the error would borrow `text`, which dies here
   |     ^^^^^^-----^
   |     |     |
   |     |     `text` is borrowed here
   |     returns a value referencing data owned by the current function
```

"`text` is borrowed here", and the error returned would be "referencing data owned by the current function". Look again at the signature: by elision, the `'_` in `ParseError<'_>` is the lifetime of `name`, the only reference parameter. The signature promises an error that borrows from `name`, and no body could keep that promise with an error that points into `text`. There is no lifetime the author could write instead, because `text` does not exist outside the function.

The fix is an owned error at the boundary where the input is created:

```
/// Owns its data: it can outlive the input, cross threads, and become a
/// `Box<dyn Error>` (which means `Box<dyn Error + 'static>`).
#[derive(Debug)]
struct ConfigError {
    source_name: String,
    line_no: usize,
    line: String,
}

impl fmt::Display for ConfigError {
    fn fmt(&self, f: &mut fmt::Formatter<'_>) -> fmt::Result {
        let (name, line_no, line) = (&self.source_name, self.line_no, &self.line);
        write!(f, "{name}:{line_no}: not a `key = value` line: {line:?}")
    }
}

impl Error for ConfigError {}
```

```
/// Reads the text itself, so nothing it returns may borrow from that text.
fn load(name: &str) -> Result<Vec<(String, String)>, ConfigError> {
    let text = read_source(name);
    let pairs = parse(&text).map_err(|e| ConfigError {
        source_name: name.to_owned(),
        line_no: e.line_no,
```

```
            line: e.line.to_owned(),
        })?;
        Ok(pairs
            .into_iter()
            .map(|(k, v)| (k.to_owned(), v.to_owned()))
            .collect())
}
```

```
// The caller owns the input: a borrowed error is fine here.
let input = String::from("a = 1\nb 2");
if let Err(e) = parse(&input) {
    println!("line {}: {:?}", e.line_no, e.line);
}

println!("{:?}", load("good.conf"));
let error: Box<dyn Error> = load("bad.conf").unwrap_err().into();
println!("{error}");
```

```
line 2: "b 2"
Ok([("name", "indexer"), ("workers", "4")])
bad.conf:2: not a `key = value` line: "workers 4"
```

`load` converts the borrowed `ParseError` into an owned `ConfigError` with `map_err`, copying one line of text, and only on the error path. The owned error can then go anywhere: here it becomes a `Box<dyn Error>`, which requires `'static` (Chapter 7; Chapter 8 shows what happens with a borrowed error and `?`).

As a rule, errors in public APIs should own their data. When copying text into the error is too expensive, store *positions* instead (a line number, a byte range): the error has no lifetime, and whoever holds the input can show the text (exercise 2). Low-level, performance-sensitive parsers may return borrowed errors, ideally with an `into_owned` method like the tokens above.

14.9 Builders

Builders configure a value step by step. They rarely need lifetime parameters, but the way their methods take `self` decides how callers can use them. The program is `code/examples/ch14_builders.rs`.

The simplest style takes the builder by value and returns it:

```
/// Each method takes the builder by value and returns it.
struct RequestBuilder {
    request: Request,
}

impl RequestBuilder {
    fn new(url: impl Into<String>) -> Self {
        let request = Request {
            url: url.into(),
            headers: Vec::new(),
            timeout_ms: 30_000,
        };
        RequestBuilder { request }
    }

    fn header(mut self, name: impl Into<String>, value: impl Into<String>) -> Self {
        self.request.headers.push((name.into(), value.into()));
        self
    }
```

```
    fn timeout_ms(mut self, timeout_ms: u64) -> Self {
        self.request.timeout_ms = timeout_ms;
        self
    }

    fn build(self) -> Request {
        self.request
    }
}
```

```
let builder = RequestBuilder::new("https://example.com/items").timeout_ms(500);
let builder = builder.header("accept", "text/plain"); // stored, then continued
let request = builder.build();
println!(
    "{} {:?} {}ms",
    request.url, request.headers, request.timeout_ms
);
```

There are no borrows here at all: each call consumes the builder and returns it, so it can be chained, stored in a variable, and continued later.

The other style, used by `std::process::Command` (whose `arg`, `env` and `current_dir` take `&mut self` and return `&mut Command`), borrows the builder:

```
/// Each method borrows the builder mutably and returns the same borrow.
struct QueryBuilder {
    table: String,
    filters: Vec<String>,
}

impl QueryBuilder {
    fn new(table: impl Into<String>) -> Self {
        QueryBuilder {
            table: table.into(),
            filters: Vec::new(),
        }
    }

    fn filter(&mut self, condition: impl Into<String>) -> &mut Self {
        self.filters.push(condition.into());
        self
    }

    fn build(&self) -> String {
        if self.filters.is_empty() {
            return format!("SELECT * FROM {}", self.table);
        }
        format!(
            "SELECT * FROM {} WHERE {}",
            self.table,
            self.filters.join(" AND ")
        )
    }
}
```

```
// One statement: the temporary builder lives until the end of the statement.
let all = QueryBuilder::new("users").filter("age > 30").build();
println!("{all}");

// Several statements: bind the builder to a variable first.
let only_active = true; // imagine a command-line flag
let mut query = QueryBuilder::new("users");
query.filter("age > 30");
if only_active {
```

```
    query.filter("active");
}
println!("{}", query.build());
```

```
https://example.com/items [("accept", "text/plain")] 500ms
SELECT * FROM users WHERE age > 30
SELECT * FROM users WHERE age > 30 AND active
```

Its advantages are that a builder can be configured conditionally without rebinding it, and reused to build several values (`build(&self)`). Its pitfall is the return type: `filter` returns a borrow of the builder. In a single statement that is fine, because a temporary lives until the end of the statement it is created in (Chapter 5). Binding the borrow to a variable is not, as `code/compile_fail/ch14_builder_temporary.rs` shows:

```
pub fn caller() -> String {
    let query = QueryBuilder::new("users").filter("age > 30");
    query.filter("active");
    query.build()
}
```

```
error[E0716]: temporary value dropped while borrowed
  --> compile_fail/ch14_builder_temporary.rs:34:17
   |
34 |     let query = QueryBuilder::new("users").filter("age > 30");
   |                 ^^^^^^^^^^^^^^^^^^^^^^^^^^                    - temporary value is freed
at the end of this statement
   |                 |
   |                 creates a temporary value which is freed while still in use
35 |     query.filter("active");
   |     ----- borrow later used here
   |
help: consider using a `let` binding to create a longer lived value
   |
34 ~     let mut binding = QueryBuilder::new("users");
35 ~     let query = binding.filter("age > 30");
   |
```

E0716, "temporary value dropped while borrowed". The temporary is the builder created by `QueryBuilder::new("users")` ("creates a temporary value which is freed while still in use"); `query` is a `&mut QueryBuilder` pointing into it; the temporary is freed at the semicolon; and `query` is used on the next line ("borrow later used here"). The help shows the fix: bind the builder itself to a variable, as the working program does.

Both builders own their data (`impl Into<String>`), which is why neither has a lifetime parameter. A builder may borrow its inputs when it is always used within one expression, but the value it builds is usually kept, so it should own.

14.10 A refactoring walkthrough: from lifetime soup to a clear design

Lifetime problems in real code rarely come one at a time. A common history is this: a struct borrows something, the compiler complains, a lifetime is added, the compiler complains elsewhere, the same lifetime is added there too, and so on, until the code compiles. The result is "lifetime soup": many

parameters, each added to silence one error, with relationships nobody designed. This section takes one such design, a small log analyzer, and improves it in three steps. Every version is a complete program, and the three that compile print the same report.

Version 1: lifetime soup

Here is the starting point, `code/compile_fail/ch14_refactor_v1.rs`:

```
pub struct Config<'c> {
    pub min_level: Level,
    pub ignore: Vec<&'c str>,
}

pub struct Analyzer<'a, 'b, 'c> {
    config: &'b Config<'c>,
    input: &'a str,
    counts: BTreeMap<Level, usize>,
    errors: Vec<&'a str>,
}

impl<'a, 'b, 'c> Analyzer<'a, 'b, 'c> {
    pub fn new(config: &'b Config<'c>, input: &'a str) -> Analyzer<'a, 'b, 'c> {
        Analyzer {
            config,
            input,
            counts: BTreeMap::new(),
            errors: Vec::new(),
        }
    }

    pub fn run(&'a mut self) {
        for line in self.input.lines() {
            let Some((level, message)) = parse_line(line) else {
                continue;
            };
            let ignored = self.config.ignore.iter().any(|word| message.contains(word));
            if level < self.config.min_level || ignored {
                continue;
            }
            *self.counts.entry(level).or_insert(0) += 1;
            if level == Level::Error {
                self.errors.push(message);
            }
        }
    }

    pub fn counts(&'a self) -> &'a BTreeMap<Level, usize> {
        &self.counts
    }

    pub fn errors(&'a self) -> &'a [&'a str] {
        &self.errors
    }
}
```

Three lifetime parameters: `'a` for the input, `'b` for the borrow of the configuration, `'c` for the strings inside the configuration. The receivers `&'a mut self` and `&'a self` were added because the methods deal with `'a` data. The library compiles. Its first caller does not:

```
pub fn caller(input: &str) {
    let config = Config {
        min_level: Level::Warn,
        ignore: vec!["heartbeat"],
    };
    let mut analyzer = Analyzer::new(&config, input);
```

```
    analyzer.run();
    println!("{:?}", analyzer.counts());
}
```

```
error[E0502]: cannot borrow `analyzer` as immutable because it is also borrowed as mutable
  --> compile_fail/ch14_refactor_v1.rs:82:22
   |
81 |     analyzer.run();
   |     -------- mutable borrow occurs here
82 |     println!("{:?}", analyzer.counts());
   |                      ^^^^^^^^
   |                      |
   |                      immutable borrow occurs here
   |                      mutable borrow later used here
```

The labels are the hallmark from the previous section: the mutable borrow of `run` is "later used" at the very next use of `analyzer`. `run(&'a mut self)` in `impl<'a, 'b, 'c> Analyzer<'a, 'b, 'c>` is `&'a mut T<'a>`, and the analyzer is borrowed forever after its first run. The worst part is *where* the error appears: not in the library, which compiled, but in the code of its users.

Version 2: fix the receivers

The minimal fix gives every receiver its own, elided lifetime (`code/examples/ch14_refactor_v2.rs`):

```
// `run` is now `fn run(&mut self)`: a short borrow, chosen at each call.
fn counts(&self) -> &BTreeMap<Level, usize> {
    &self.counts
}

fn errors(&self) -> &[&'a str] {
    &self.errors
}
```

```
let input = String::from(LOG);
let config = Config {
    min_level: Level::Warn,
    ignore: vec!["heartbeat"],
};
let mut analyzer = Analyzer::new(&config, &input);
analyzer.run();
for (level, count) in analyzer.counts() {
    println!("{level:?}: {count}");
}
println!("errors: {:?}", analyzer.errors());
```

```
Warn: 1
Error: 2
errors: ["db timeout after 30s", "disk full"]
```

`errors(&self) -> &[&'a str]` shows the two borrows clearly: the slice borrows the analyzer (briefly, elided), and the strings in it borrow the input (`'a`). This version is correct. But look at what remains: three lifetime parameters, and two of them buy nothing. The configuration is a few bytes, built once; borrowing it forces every analyzer to live inside the configuration's scope, and `'c` makes the configuration itself a view of string literals. Anyone who stores an `Analyzer` in a struct must now write three lifetimes.

Version 3: own what is small, borrow what is big

The input can be large, and borrowing it is the point of the design. The configuration is small: let it own its strings, and borrow it only for the call that needs it (`code/examples/ch14_refactor_v3.rs`):

```
/// Small, built once, read often: it owns its data.
struct Config {
    min_level: Level,
    ignore: Vec<String>,
}

/// Borrows the log it analyzes, and nothing else.
struct Analyzer<'src> {
    input: &'src str,
    counts: BTreeMap<Level, usize>,
    errors: Vec<&'src str>,
}

impl<'src> Analyzer<'src> {
    fn new(input: &'src str) -> Self {
        Analyzer {
            input,
            counts: BTreeMap::new(),
            errors: Vec::new(),
        }
    }

    /// The configuration is needed only during the call: borrow it there.
    fn run(&mut self, config: &Config) {
        for line in self.input.lines() {
            let Some((level, message)) = parse_line(line) else {
                continue;
            };
            let ignored = config
                .ignore
                .iter()
                .any(|word| message.contains(word.as_str()));
            if level < config.min_level || ignored {
                continue;
            }
            *self.counts.entry(level).or_insert(0) += 1;
            if level == Level::Error {
                self.errors.push(message);
            }
        }
    }

    fn counts(&self) -> &BTreeMap<Level, usize> {
        &self.counts
    }

    fn errors(&self) -> &[&'src str] {
        &self.errors
    }
}
```

```
Warn: 1
Error: 2
errors: ["db timeout after 30s", "disk full"]
```

One lifetime parameter is left, named after what it borrows: `'src` , the log. `Config` has no lifetime, and `run(&mut self, config: &Config)` borrows it for the duration of the call, so one configuration can serve any number of analyzers.

Version 4: views for the work, owned values for the results

One question remains: why does the *analyzer* hold borrows at all? The input is needed only while the analysis runs; what is kept afterward is the result. The final version separates the two (`code/examples/ch14_refactor_v4.rs`). The work uses a zero-copy view of each line:

```
/// One parsed line: a zero-copy view into the log.
#[derive(Clone, Copy, Debug)]
struct Entry<'src> {
    level: Level,
    message: &'src str,
}

impl<'src> Entry<'src> {
    fn parse(line: &'src str) -> Option<Self> {
        let (level, message) = line.split_once(' ')?;
        let level = match level {
            "INFO" => Level::Info,
            "WARN" => Level::Warn,
            "ERROR" => Level::Error,
            _ => return None,
        };
        Some(Entry { level, message })
    }
}

/// The entries of a log, borrowed from it.
fn entries(input: &str) -> impl Iterator<Item = Entry<'_>> {
    input.lines().filter_map(Entry::parse)
}
```

and the result owns its data:

```
struct Config {
    min_level: Level,
    ignore: Vec<String>,
}

impl Config {
    fn accepts(&self, entry: &Entry<'_>) -> bool {
        let ignored = self
            .ignore
            .iter()
            .any(|word| entry.message.contains(word.as_str()));
        entry.level >= self.min_level && !ignored
    }
}

/// What we keep: it owns its data, so it can outlive the log.
#[derive(Default)]
struct Report {
    counts: BTreeMap<Level, usize>,
    errors: Vec<String>,
}

/// Borrows the log and the configuration for the duration of the call only.
fn analyze(input: &str, config: &Config) -> Report {
    let mut report = Report::default();
    for entry in entries(input).filter(|entry| config.accepts(entry)) {
        *report.counts.entry(entry.level).or_insert(0) += 1;
        if entry.level == Level::Error {
            report.errors.push(entry.message.to_owned());
        }
    }
    report
}
```

```
let input = String::from(LOG);
let config = Config {
    min_level: Level::Warn,
    ignore: vec![String::from("heartbeat")],
};
let report = analyze(&input, &config);
drop(input); // the report does not borrow the log
for (level, count) in &report.counts {
    println!("{level:?}: {count}");
}
println!("errors: {:?}", report.errors);

// `Report: Send + 'static`: it can be handed to another thread.
let handle = thread::spawn(move || report.errors.len());
assert_eq!(handle.join().unwrap(), 2);
```

```
Warn: 1
Error: 2
errors: ["db timeout after 30s", "disk full"]
```

`analyze` is the boundary. It borrows the log and the configuration for the duration of one call, works with borrowed `Entry<'src>` values that never leave it, and returns a `Report` that owns everything it contains. The `main` function drops the input before printing the report, and sends the report to another thread afterward: neither would compile if the report borrowed the log. The price is a copy of each error message, a few bytes; if it ever mattered, the report could store line numbers instead.

What changed

Version	Lifetime parameters	What it means for users
1	three, and `&'a` receivers	the analyzer is unusable after `run` (E0502)
2	three	works; every user writes three lifetimes
3	one, `'src`	the analyzer borrows the log only
4	none on kept types	`Report` can be stored, returned, sent

The general recipe, in the order the walkthrough applied it:

1. Remove lifetimes from receivers: `&self` and `&mut self` almost never need a named lifetime. A lifetime on both a `&mut` and the type behind it is a bug.
2. Own small data, especially configuration. Borrow it per call if needed.
3. Name the lifetimes that remain after what they borrow.
4. Borrow in the functions that do the work; own in the types that hold the results. A function call is a natural boundary between the two.

14.11 Common confusions

"**More lifetime annotations make code faster or more precise.**" Lifetimes are erased before code generation (Chapter 1); they never make code faster. Each annotation is a constraint, and a wrong one, like `&'a mut self`, only rejects more programs.

"**Owning is slow, so borrow everywhere.**" Copying a configuration value or an error message at a boundary costs almost nothing. Borrowing pays off where data is large or produced in great numbers, such as the tokens of a big input. Borrow there, own elsewhere, and measure before trading clarity for speed.

"**A method that stores `&'a` data needs `&'a mut self`.**" It needs the *data* to be `&'a`, which is a requirement on the argument. The receiver's borrow should stay short and elided.

"**One lifetime parameter is always simpler.**" Simpler to write, yes. When a struct's results borrow from only one of its sources, or when it holds a `&mut` to a container of borrows, one parameter is wrong, and the error shows up in the callers.

"**`Cow` everywhere avoids every decision.**" `Cow` adds a lifetime parameter (or a `'static` restriction) and a branch at every use. It is the right type when both cases, borrowed and owned, are common; otherwise pick one.

"**A callback that receives `&str` can keep it if the data lives long enough.**" Only if the signature says so. With `impl Fn(&str)`, the callback must work for any lifetime, so it can never keep the borrow, however long the data actually lives.

14.12 Summary

- Borrow for the call, own for storage. Parameters that are only read are `&str`, `&[T]` or `impl AsRef<str>`; parameters that are kept are `String` or `impl Into<String>`, which lets callers move a `String` in without a copy.
- Return a reference for a part of an input, an owned value for new data, and `Cow<'a, str>` when it can be either. Structs that are kept should own their fields; structs with reference fields are views, for short-lived work.
- `Cow<'static, str>` holds a literal or a run-time string without a lifetime parameter. `Arc<str>` and `Rc<str>` share immutable text among many owners, with no lifetime parameter and cheap clones.
- Zero-copy designs pair a borrowed type (`Token<'src>`) for the work with an owned twin (`OwnedToken`, via `into_owned`) for what is kept.
- One lifetime parameter is usually enough, thanks to covariance. Use two when results borrow from only one of two sources, or when a field is a `&mut` to a container of borrows. Name lifetimes after what they borrow.
- `&'a mut T<'a>` borrows a value for the rest of its life ("borrowed forever"). It appears as `&'a mut self` in `impl<'a> T<'a>` and as `&'a mut Vec<&'a str>` in fields. Give the outer borrow its own lifetime.
- `impl Fn(&T)` callbacks receive borrows they cannot keep (E0521); a signature with a named lifetime lets them keep the borrow, at the cost of a stronger promise. Stored callbacks are `'static` by default; `dyn Fn() + 'a` lets them borrow.

- `impl Iterator<Item = &T>` captures the `&self` lifetime in edition 2024; use a named iterator type when callers need to store it, and `use<'src>` when the items borrow from a source rather than from `self`.
- Public errors should own their data (or store positions). Builders should own their data; `&mut self` builders must be bound to a variable before being configured over several statements (E0716).

14.13 Exercises

1. In a copy of `ch14_tokenizer.rs`, give `Token<'src>` fields of type `Cow<'src, str>` instead of `&'src str`, and a method `fn into_owned(self) -> Token<'static>`. Then remove `OwnedToken`: the history becomes a `Vec<Token<'static>>`. *Hint:* each variant becomes `Ident(Cow<'src, str>)`; the tokenizer creates `Cow::Borrowed(..)`, and `into_owned` rebuilds each variant with `Cow::Owned(text.into_owned())`.
2. In a copy of `ch14_errors.rs`, make `ParseError` store the byte range of the offending line, a `std::ops::Range<usize>`, instead of a `&'src str`. Add a method `line` that takes the source and returns the text of the line. Can `load` now return a `ParseError` directly? Is that a good idea? *Hint:* the method is `fn line<'s>(&self, src: &'s str) -> &'s str`. The error has no lifetime parameter any more, so the E0515 of `ch14_error_borrows_local.rs` disappears. But the caller of `load` never sees the text, so the range alone is not very useful to it.
3. Change `for_each_line` in `ch14_callbacks.rs` so that it passes each line in lowercase, built in a `String` buffer that is reused for every line. Then try the same change in `for_each_line_kept`. *Hint:* the first signature allows it, because the callback accepts a borrow of any lifetime, including a borrow of a local buffer. The second promised `&'text str`, and a borrow of the buffer is not one: the compiler rejects `f(&buffer)` (E0597, "`buffer` does not live long enough").
4. Give `WordSet` from `ch14_borrowed_forever_fixed.rs` a field `counts: BTreeMap<&'text str, usize>` and a method `fn most_common(&self) -> Option<&'text str>`. Show that the result can be used after the `WordSet` is dropped. *Hint:* the result is copied out of the map with the lifetime `'text`, not the lifetime of `&self`; write `self.counts.iter().max_by_key(..).map(|(w, _)| *w)`.
5. Extend version 4 of the log analyzer: add `fn merge(&mut self, other: Report)` to `Report`, and analyze two logs on two threads with `thread::scope`, merging the reports in `main`. Which of the four versions could you have done this with `thread::spawn` instead? *Hint:* scoped threads may borrow the logs and the configuration (Chapter 7). With `thread::spawn`, the threads must own their inputs, and only version 4's `Report` can come back from a thread on its own.